

Database Migration

Best Practices for PostgreSQL, MySQL, SQL Server & Oracle

Database VMs are the highest-risk, highest-reward migration target. This guide covers per-database strategies: when to stop vs snapshot, replication-based cutover, VirtIO I/O performance, SQL Server licensing savings with CPU pinning, and zero-downtime approaches using read replicas.

PostgreSQL | MySQL | SQL Server | Oracle | Zero-Downtime | CPU Pinning

The Database Migration Challenge

Database VMs carry the most risk — and the biggest savings opportunity.

Why databases are different from other VMs

Databases have active writes, transaction logs, replication relationships, and strict consistency requirements. A naive VM migration (shutdown → export → convert → boot) causes downtime. A crash-consistent snapshot may lose transactions. The strategy depends on the database engine and acceptable downtime window.

Migration Strategies by Downtime Tolerance

Strategy	Downtime	How It Works	Best For	Databases
Cold migration	30-60 min	Stop DB → export VM → convert → boot on KVM → start DB	Dev/test, small DBs, planned windows	All databases
Snapshot migration	5-15 min	VMware snapshot → export → convert → boot → DB crash recovery	Tolerant DBs with WAL/redo logs	PostgreSQL, MySQL InnoDB, Oracle
Replica cutover	<1 min	Set up replication to KVM replica → promote → cutover DNS	Production DBs needing near-zero downtime	PostgreSQL, MySQL, SQL Server AG
Logical migration	0 (online)	pg_dump/mysqldump or logical replication to new KVM instance	Schema + data only (no VM migration)	PostgreSQL, MySQL (not VM-level)

Per-Database Best Practices

PostgreSQL

- **Preferred:** Streaming replication to KVM replica, then promote
- **Cold:** `pg_ctl stop -m fast` before export
- **Snapshot:** Safe — WAL replay recovers from crash-consistent state
- **Post-migration:** Verify `pg_isready`, check replication lag
- **VirtIO performance:** Comparable to PVSCSI for OLTP workloads
- **Tip:** Separate WAL on second VirtIO disk for best I/O

MySQL / MariaDB

- **Preferred:** MySQL replication to KVM replica, then promote
- **Cold:** `mysqladmin shutdown` before export (clean shutdown)
- **Snapshot:** InnoDB crash recovery handles inconsistency
- **GTID replication:** Set up replica on KVM, catch up, promote
- **Post-migration:** `SHOW SLAVE STATUS`, check binary log position
- **Tip:** `innodb_flush_log_at_trx_commit=1` for safety

SQL Server

- **Preferred:** Always-On AG — add KVM replica, failover
- **Cold:** Stop SQL Server service before export
- **VirtIO injection:** hyper2kvm handles automatically
- **Licensing win:** KVM CPU pinning reduces per-core licensing 20-40%
- **Post-migration:** Verify databases online, AG health, agent jobs
- **Tip:** Lock pages in memory + large pages on KVM for performance

Oracle Database

- **Preferred:** Data Guard to KVM standby, switchover
- **Cold:** `shutdown immediate` before export
- **Snapshot:** Redo log replay recovers crash-consistent state
- **RAC:** Migrate one node at a time (rolling migration)
- **Post-migration:** Verify `SELECT STATUS FROM V$INSTANCE`
- **Tip:** ASM disk groups migrate with the VM disk

SQL Server Licensing Savings

The biggest hidden win of database migration: reduced per-core licensing costs on KVM.

The VMware SQL Server Licensing Problem

On VMware

- VMware vSphere exposes all physical cores to the guest
- Microsoft licensing may count all host cores
- A 2-socket, 16-core/socket host = 32 cores to license
- SQL Standard: $\$3,945 / 2\text{-core pack} \times 16 = \mathbf{\$63,120}$
- SQL Enterprise: $\$15,123 / 2\text{-core pack} \times 16 = \mathbf{\$241,968}$
- Even if SQL VM uses only 4 vCPUs

On KVM (with CPU pinning)

- KVM with CPU pinning exposes only assigned cores
- SQL VM with 4 pinned vCPUs = license only 4 cores
- NUMA-aware placement for performance + compliance
- cgroups enforce CPU isolation — verifiable by auditors
- SQL Standard: $\$3,945 \times 2 = \mathbf{\$7,890}$
- SQL Enterprise: $\$15,123 \times 2 = \mathbf{\$30,246}$

Savings Calculator: 10 SQL Server Instances

Edition	VMware (32 cores/host)	KVM (4 pinned vCPUs)	Per-Server Savings	10 Servers / 3 Years
SQL Standard	\$63,120	\$7,890	\$55,230	\$1,656,900
SQL Enterprise	\$241,968	\$30,246	\$211,722	\$6,351,660

Note: Actual savings depend on Microsoft licensing agreement, Software Assurance, and core count. Consult your Microsoft licensing advisor for exact numbers.

Zero-Downtime Database Migration with Replication

```
# PostgreSQL: Streaming replication to KVM target
# 1. Set up KVM VM with PostgreSQL (same version)
# 2. Configure streaming replication from VMware source
# 3. Wait for replica to catch up (lag = 0)
# 4. Application maintenance mode (1 minute)
# 5. Promote KVM replica: SELECT pg_promote()
# 6. Update DNS/connection strings to KVM
# 7. Verify application connectivity
# Total downtime: <1 minute
```

Database Migration: Highest Risk, Highest Reward

Use replication for near-zero downtime. Use CPU pinning for SQL Server licensing savings.
hyper2kvm handles the VM-level conversion (VirtIO, fstab, initramfs) automatically.

The database strategy is your choice. The VM migration is automated.